Week 18 | From next-character likelihood to causal wake decoding

Why a language-model lesson belongs in this course

The aim is to teach the connection requested for the LLM extension: a model represents context and learns a conditional distribution or a conditional state transition. A fluid Transformer is not automatically a language model, and calling a sequence model causal does not explain how language-model training works. This required laboratory trains a small character model and then transfers the masking and shifted-target reasoning to continuous POD tokens.

The language corpus consists of actual FlowMLLab documentation: the modal-data contract, results guide and Week 7.3 notes. A separate Week 7.4 document supplies validation text. Vocabulary comes only from training documents, and unseen validation characters map to an explicit unknown token. This document split is stronger than withholding positions from repeated templates, but shared course vocabulary and short context still limit general-language claims. Corpus paths and hashes are printed in the notebook.

Autoregressive likelihood and teacher forcing

For discrete tokens, the chain rule factors the joint sequence probability into conditional next-token distributions. A neural network supplies logits for every position. Cross-entropy is the average negative log probability assigned to the observed next token. Padding, if introduced, must be excluded from the average.

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$$

$$\mathrm{L} = -\frac{1}{T}\sum_{t} \log p_\theta(x_{t+1} \mid x_{\le t})$$

Teacher forcing supplies the observed prefix during fitting. At inference the generated token is fed back into the context. Parallel training is possible because every shifted target is available in the batch, but that convenience creates a leakage route: the representation of an early position must not read the later input token that reveals its target.

A length-four input (a,b,c,d) is paired with targets (b,c,d,e). The target b is physically present elsewhere in the input tensor. A causal mask blocks access from position zero to that later input; simply shifting the target array is insufficient. A split boundary also matters: training windows must be formed inside the training partition, not formed globally and split afterward.

The mask and its decisive counterexample

A causal self-attention score above the main diagonal is replaced by negative infinity before softmax. Thus output position t can depend only on positions at or before t. A prefix-invariance test changes the future tokens and requires earlier outputs to remain unchanged. The deliberately unmasked control must fail that test for a nondegenerate input and model.

$$M_{ij} = 0 \ (j \le i), \qquad M_{ij} = -\infty \ (j > i)$$

The earlier one-block temporal model read only the final output token. The final mask row excludes nothing, so removing that mask had no effect on its prediction. That model could still be causal because its input window contained only past states; it simply could not demonstrate masked parallel next-token learning. The new decoder predicts every position, uses two blocks, and is tested on intermediate outputs.

The test establishes a dependency property. It does not guarantee that an unmasked model always has lower training loss or that every unmasked rollout must fail. Those are empirical outcomes depending on optimization and data. The acceptance criterion is the controlled future-perturbation experiment, not a predetermined performance ranking.

A worked likelihood calculation

Suppose the correct next character receives probabilities 0.8, 0.5 and 0.25 at three positions. The average negative natural logarithm is about 0.7675, and perplexity is exp(0.7675), about 2.154. Computing the average probability first and taking its reciprocal gives a different quantity and is not perplexity.

$$\mathrm{PPL} = \exp(\mathrm{L})$$

Perplexities from different vocabularies or tokenization schemes are not directly comparable. A character vocabulary of roughly two dozen symbols and a subword vocabulary of tens of thousands define different prediction events. Likewise, low loss on course documentation does not certify correct physical facts in sampled text. Ask the model to generate, inspect the sample, and distinguish fluent continuation from verified numerical knowledge.

Sampling and context windows

Temperature divides logits before normalization. It changes the distribution, not the learned parameters. Lower temperature usually reduces diversity; it can also repeatedly choose an incorrect high-probability token. Sampling uses a declared random seed in the laboratory so students can compare implementations without mistaking random variation for a scientific finding.

The model only sees the last context-length tokens. When a long generated record slides out of the window, information needed to maintain its structure may disappear. A failure after a context shift is an opportunity to inspect the input actually supplied to the model. It should not be hidden by presenting only the first plausible sentence.

The notebook prints the complete short sample, vocabulary size, held-out cross-entropy and training curve. The sample is explicitly labeled unvalidated generated text. Students vary temperature and starting prefix, then explain one failure using the context they reconstructed from the sampling loop.

Mapping discrete language to continuous flow

Language-model concept	Flow analogue	Important difference
Character or subword token	One POD state vector at a time	Continuous vector, not a vocabulary index
Embedding lookup	Linear projection of coefficients	Magnitude and normalization matter
Position representation	Time position within the context	Uniform sampling is assumed here
Next-token likelihood	Next-state regression loss	MSE is not token cross-entropy
Causal mask	No future state in intermediate outputs	Also enforce chronological window construction
Teacher forcing	Observed past flow states	Deployment feeds back predictions
Context window	Four-frame history	Physical duration depends on time step
Pretraining	Source-trajectory fitting	Source data and compute must be counted
Fine-tuning	Target-trajectory adaptation	Target labels include model selection

With a fixed isotropic Gaussian observation model, minimizing squared error corresponds to maximizing a continuous conditional likelihood up to constants. That interpretation requires a specified variance and does not make a deterministic MSE predictor a calibrated probabilistic forecaster.

Exposure bias and the classroom experiment

One-step accuracy is measured under observed histories. Autonomous rollout is measured under histories partly or entirely produced by the model. Their input distributions differ. Scheduled sampling mixes predicted and observed inputs during training, but it changes the fitting procedure and should be studied as a declared ablation, not silently added to one model only.

The integrated exercise computes language-model NLL and accuracy at every position, alongside overall NLL and perplexity. A predictor that is correct only at the final token must still show errors at earlier positions. The worked wake example supplies shifted all-position regression targets; the coding task diagnoses supervision with an explicit language likelihood rather than a final-token score.

Deliverables and reading

Submit shifted-window tests, a hand-checked cross-entropy example, the masked/unmasked prefix experiment, one sampled-text failure analysis and the language-to-flow mapping. Use the four executable coding checks plus a written explanation of teacher forcing versus autonomous deployment. Week 21 will evaluate that deployment gap using field and phase diagnostics.

Primary architecture reference: Vaswani et al. (2017), https://arxiv.org/abs/1706.03762. Implementation reading: Raschka, https://github.com/rasbt/LLMs-from-scratch. The corpus is attributed to the existing FlowMLLab documentation; the experiments are original and no published large model is reproduced here.

Printable laboratory worksheet

Task 1: Shift targets

Build next-character input/target windows without allowing a window to cross the provided sequence boundary.

Interface contract: Input a is a one-dimensional NumPy array, with 1<=k<len(a). Return a pair of arrays with shape [len(a)-k,k], including every stride-one window.

Task 2: Cross-entropy and perplexity

Compute negative log likelihood from logits using log_softmax; average over all positions.

Interface contract: Inputs logits:[batch,time,vocabulary] and integer targets:[batch,time] are Torch tensors. Return one scalar Torch tensor containing mean natural-log NLL.

Task 3: Causal prefix invariant

Return whether future perturbation leaves the prefix unchanged. Test both the masked model and its deliberately broken control.

Interface contract: model returns a [batch,time,feature] tensor and is already in evaluation mode. x has that shape; 0<prefix<time. Add 10 to all future input positions, hold the prefix fixed, and return a Python bool comparing every prefix output at atol=1e-6.

Task 4: Evaluate a language model by position

Return per-position negative log likelihood, token accuracy, overall NLL and perplexity for [batch,time,vocabulary] logits. Validate dimensions and targets. Compare a correct predictor with one that is correct only at the final position; report why final-token accuracy can hide training failures.

Interface contract: Return a dictionary with position_nll and position_accuracy as length-time Torch tensors averaged over batch, nll and perplexity as Python floats averaged over all tokens. Use natural logs. Non-3-D logits, mismatched target shape, empty targets or tokens outside [0,vocabulary) must raise ValueError. Targets are integer token IDs.

Before submitting

Restart the kernel and run the worked examples and completed tasks. Confirm that all four coding checks pass and that no retained data or results file changed. Include the requested interpretation, identify every quantity fitted from training data, and separate test observations from decisions made using validation. Record unresolved budget or representation limits rather than removing inconvenient runs.

Assessment and worked interpretation

Coding: Tasks 1–3 carry 5 points each; the integrated Task 4 carries 25. Derivation (25): trace a 4-head causal Block and shifted targets. Interpretation (25): give evidence and limitations. Reproducibility (10): identify inputs, split, checkpoint, environment and every failed seed. Total 100. Passing function checks earns only the coding allocation.

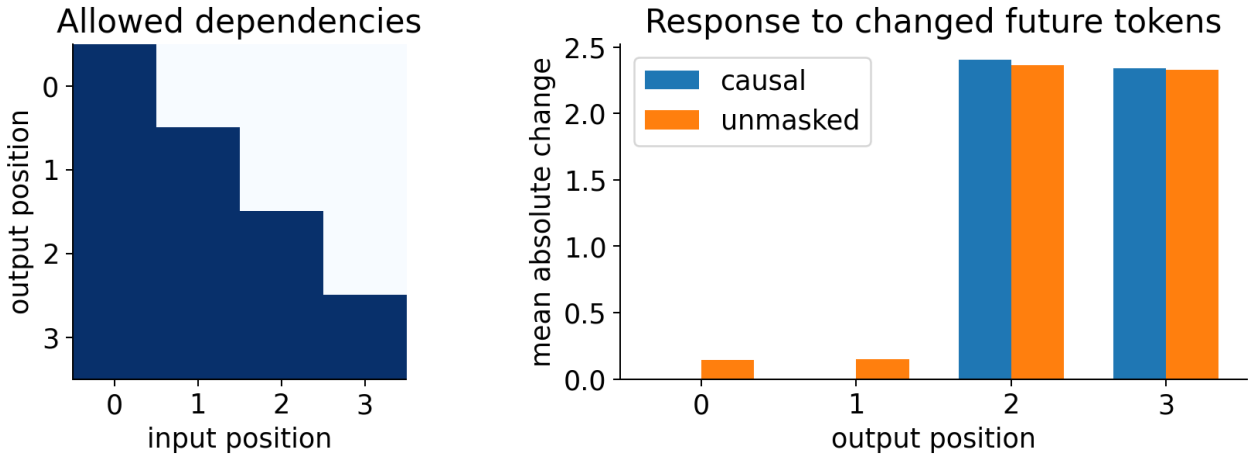
Worked full-credit reasoning: Masking blocks access to future tokens; the poisoned-prefix test establishes a dependency property, not forecast accuracy. Window CE covers every validation window and includes repeated characters.

Misconception to reject: “Low character perplexity proves a stable CFD rollout.” Explain the failure in one equation or counterexample. An observation without protocol and limitation earns at most half of interpretation credit. Instructor judgment and a timed student pilot remain necessary.

The language-model validation metric sums next-token cross-entropy over every held-out window and divides by evaluated tokens. Overlap repeats characters; it is not a unique-character document likelihood. The vocabulary embedding, position vectors, four width-6 causal heads and feed-forward residual are traced in the notebook; the continuous CFD experiment is separate.

Timing estimate: a 120-minute guided lab and roughly 3–5 hours including preparation and written analysis. This is an editorial estimate; a timed student pilot remains open.

Executed evidence and interpretation



This figure is generated from the retained protocol. Use the notebook to inspect all seeds and recompute the plotted quantities. The earlier lecture sections specify what the diagnostic can and cannot establish.